

10_tarea_heatmap_categorico_videogames

April 27, 2026

1 Tarea paso a paso: Heatmap Categórico

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Heatmaps Categóricos**. Su objetivo es que practiques cómo cruzar dos variables categóricas y representar su relación numérica mediante una cuadrícula de colores, detectando patrones que con un gráfico de barras serían imposibles de ver.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Seaborn, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset **“Video Game Sales”**, que recoge las ventas globales de más de 16.000 videojuegos desde 1980 hasta 2016. Cada fila representa un juego con su plataforma (PS4, Xbox, PC...), género (Action, Sports, RPG...) y ventas en millones de unidades en distintas regiones.

Es un dataset perfecto para un **Heatmap Categórico** porque cruzar **Género** con **Plataforma** genera una tabla de doble entrada que sería ilegible como gráfico de barras múltiples, pero que el heatmap convierte en un vistazo inmediato.

1. Descarga el dataset de Kaggle: [Video Game Sales](#)
2. Guarda el archivo `vgsales.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# CARGA DE DATOS
df = pd.read_csv("data/vgsales.csv")

df.head(5)
```

```
[1]:
```

	Rank	Name	Platform	Year	Genre	Publisher	\
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	

	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	41.49	29.02	3.77	8.46	82.74
1	29.08	3.58	6.81	0.77	40.24
2	15.85	12.88	3.79	3.31	35.82
3	15.75	11.01	3.28	2.96	33.00
4	11.27	8.89	10.22	1.00	31.37

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:
```

	Rank	Name	Platform	Year	\
count	16598.000000	16598	16598	16327.000000	
unique	NaN	11493	31	NaN	
top	NaN	Need for Speed: Most Wanted	DS	NaN	
freq	NaN	12	2163	NaN	
mean	8300.605254	NaN	NaN	2006.406443	
std	4791.853933	NaN	NaN	5.828981	
min	1.000000	NaN	NaN	1980.000000	
25%	4151.250000	NaN	NaN	2003.000000	
50%	8300.500000	NaN	NaN	2007.000000	
75%	12449.750000	NaN	NaN	2010.000000	
max	16600.000000	NaN	NaN	2020.000000	

	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	\
count	16598	16540	16598.000000	16598.000000	16598.000000	
unique	12	578	NaN	NaN	NaN	
top	Action	Electronic Arts	NaN	NaN	NaN	
freq	3316	1351	NaN	NaN	NaN	
mean	NaN	NaN	0.264667	0.146652	0.077782	
std	NaN	NaN	0.816683	0.505351	0.309291	
min	NaN	NaN	0.000000	0.000000	0.000000	
25%	NaN	NaN	0.000000	0.000000	0.000000	
50%	NaN	NaN	0.080000	0.020000	0.000000	
75%	NaN	NaN	0.240000	0.110000	0.040000	
max	NaN	NaN	41.490000	29.020000	10.220000	

	Other_Sales	Global_Sales
count	16598.000000	16598.000000
unique	NaN	NaN
top	NaN	NaN

freq	NaN	NaN
mean	0.048063	0.537441
std	0.188588	1.555028
min	0.000000	0.010000
25%	0.000000	0.060000
50%	0.010000	0.170000
75%	0.040000	0.470000
max	10.570000	82.740000

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:
```

	Rank	Name	Platform	\
0	1	Wii Sports	Wii	
1	2	Super Mario Bros.	NES	
2	3	Mario Kart Wii	Wii	
3	4	Wii Sports Resort	Wii	
4	5	Pokemon Red/Pokemon Blue	GB	
...	...	...	...	
16593	16596	Woody Woodpecker in Crazy Castle 5	GBA	
16594	16597	Men in Black II: Alien Escape	GC	
16595	16598	SCORE International Baja 1000: The Official Game	PS2	
16596	16599	Know How 2	DS	
16597	16600	Spirits & Spells	GBA	

	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	\
0	2006.0	Sports	Nintendo	41.49	29.02	3.77	
1	1985.0	Platform	Nintendo	29.08	3.58	6.81	
2	2008.0	Racing	Nintendo	15.85	12.88	3.79	
3	2009.0	Sports	Nintendo	15.75	11.01	3.28	
4	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	
...	...	...	...	...	...	...	
16593	2002.0	Platform	Kemco	0.01	0.00	0.00	
16594	2003.0	Shooter	Infogrames	0.01	0.00	0.00	
16595	2008.0	Racing	Activision	0.00	0.00	0.00	
16596	2010.0	Puzzle	7G//AMES	0.00	0.01	0.00	
16597	2003.0	Platform	Wanadoo	0.01	0.00	0.00	

	Other_Sales	Global_Sales
0	8.46	82.74
1	0.77	40.24
2	3.31	35.82
3	2.96	33.00
4	1.00	31.37
...	...	...
16593	0.00	0.01

16594	0.00	0.01
16595	0.00	0.01
16596	0.00	0.01
16597	0.00	0.01

[16291 rows x 11 columns]

1.2 Paso 1: Preparar la Tabla Pivote (Género × Plataforma)

Objetivo: Transformar el DataFrame en una tabla de doble entrada (pivote) que cruce **Genre** y **Platform** sumando las ventas globales. Esta tabla es la que alimentará el heatmap.

Instrucciones: 1. Para no generar un heatmap demasiado grande, filtra el DataFrame para quedarte solo con las **10 plataformas con más ventas totales**. Para ello: - Calcula las ventas totales por plataforma: `df.groupby('Platform')['Global_Sales'].sum()`. - Obtén los nombres de las 10 plataformas top con `.nlargest(10).index`. - Filtra el DataFrame: `df_filtrado = df[df['Platform'].isin(top_plataformas)]`. 2. Agrupa `df_filtrado` por `['Genre', 'Platform']` y calcula la **suma** de `Global_Sales`. Guarda en `agrupado`. 3. Crea la tabla pivote con `agrupado.unstack()`. Si es necesario, aplica `.droplevel(0, axis=1)` para quitar el nivel extra de columnas. 4. Muestra la tabla pivote resultante.

```
[4]: # Top 10 plataformas por ventas globales (filtro para que el heatmap no quede
      ↪ilegible)
top_plataformas = df.groupby('Platform')['Global_Sales'].sum().nlargest(10).
      ↪index
df_filtrado = df[df['Platform'].isin(top_plataformas)]

# Agrupamos datos por género y plataforma
agrupado = df_filtrado.groupby(['Genre', 'Platform']).agg({
    'Global_Sales': 'sum'          # sum porque me interesa el volumen total,
    ↪no la media
})

pivote = agrupado.unstack().droplevel(0, axis=1)
pivote
```

```
[4]: Platform      DS      GBA      PC      PS      PS2      PS3      PS4      PSP \
Genre
Action      114.16  54.26  30.67  125.74  272.43  304.02  87.06  62.66
Adventure    47.15  12.10  10.09  20.77   21.16  22.87   4.70  10.69
Fighting      7.20   4.21   0.14  72.68  89.19  51.70   8.04  21.88
Misc      137.67  28.50   8.41  44.90  98.69  45.91   7.40  13.70
Platform     77.40  78.08   0.49  64.21  72.11  29.85   7.01  17.28
Puzzle       83.87  12.09   0.92  12.08   5.90   0.46   0.02   5.52
Racing       38.58  18.80   3.80 102.89 154.21  73.10  11.53  34.73
Role-Playing 126.56  64.21  47.37  78.30  91.55  75.24  25.77  49.01
Shooter       8.20   3.60  43.44  39.31 108.28 195.80  75.32  19.77
```

Simulation	131.65	5.91	51.73	25.33	42.26	10.72	0.77	6.28
Sports	31.71	16.41	12.01	119.51	262.64	134.91	50.07	39.90
Strategy	14.76	7.45	45.63	21.67	15.04	4.77	0.41	10.29

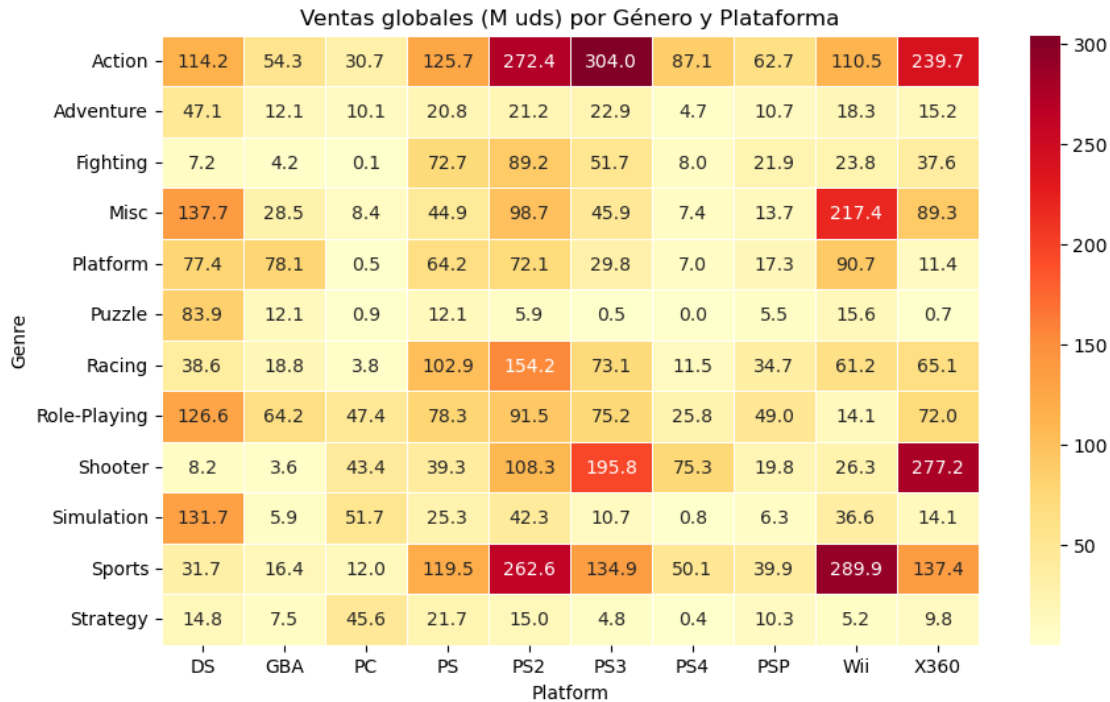
Platform	Wii	X360
Genre		
Action	110.48	239.67
Adventure	18.33	15.23
Fighting	23.82	37.64
Misc	217.43	89.33
Platform	90.68	11.39
Puzzle	15.63	0.71
Racing	61.24	65.13
Role-Playing	14.06	71.97
Shooter	26.34	277.23
Simulation	36.62	14.10
Sports	289.95	137.43
Strategy	5.23	9.77

1.3 Paso 2: Heatmap Básico con Anotaciones

Objetivo: Crear el heatmap más sencillo posible a partir de la tabla pivote, añadiendo los valores numéricos dentro de cada celda para facilitar la lectura.

Instrucciones: 1. Usa `sns.heatmap()` pasándole la tabla pivote del Paso 1. 2. Añade `annot=True` para que aparezca el valor numérico dentro de cada celda. 3. Añade `fmt='.1f'` para que los números se muestren con un decimal. 4. Añade `linewidth=0.5` para separar visualmente las celdas. 5. Muestra el gráfico. ¿Qué género y plataforma combinan las mayores ventas globales?

```
[5]: # Heatmap de variables categóricas que codifican una variable numérica
# https://seaborn.pydata.org/generated/seaborn.heatmap.html
plt.figure(figsize=(10, 6))
sns.heatmap(
    pivote,
    annot=True,
    cmap='YlOrRd',
    linewidth=0.5,
    fmt='.1f'
)
plt.title('Ventas globales (M uds) por Género y Plataforma')
plt.show()
```



1.4 Paso 3: Heatmap Temporal (Género × Década)

Objetivo: Construir un heatmap temporal que muestre cómo han evolucionado las ventas de cada género a lo largo del tiempo, agrupando los años por décadas.

Instrucciones:

1. Convierte la columna Year a entero: `df['Year'] = df['Year'].astype(int)`.
2. Crea una nueva columna Decada calculando la década de cada año: `df['Decada'] = (df['Year'] // 10) * 10`. Esto convierte 1995 → 1990, 2003 → 2000, etc.
3. Agrupa por ['Genre', 'Decada'] y calcula la suma de Global_Sales. Aplica `.unstack()` y `.droplevel(0, axis=1)` para obtener la tabla pivote. Guarda en `pivote_temporal`.
4. Crea el heatmap con `plt.figure(figsize=(10, 6))` y `sns.heatmap(pivote_temporal, cmap='YlOrRd', linewidth=0.3, annot=True, fmt='.0f')`.
5. Muestra el gráfico. ¿En qué décadas vendieron más los juegos de Action? ¿Y los de Sports?

[6]: *# Year a int y saco la década (1995 -> 1990, 2003 -> 2000, etc.)*

```
df['Year'] = df['Year'].astype(int)
df['Decada'] = (df['Year'] // 10) * 10

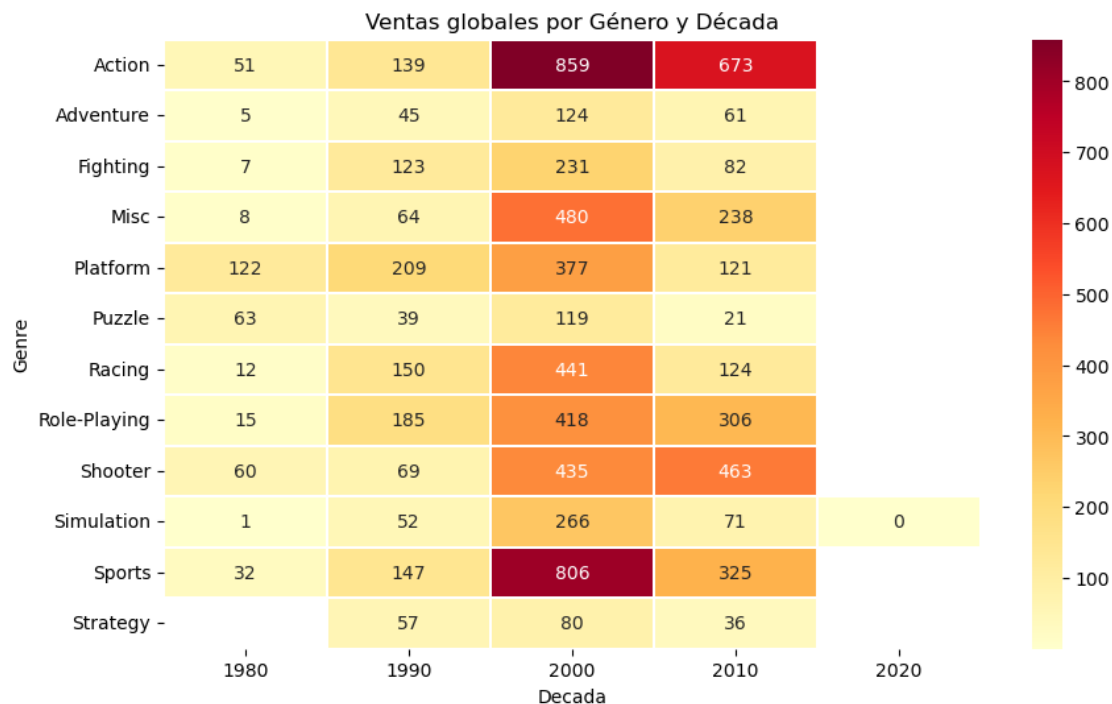
# Agrupamos datos por género y década
pivote_temporal = df.groupby(['Genre', 'Decada']).agg({
    'Global_Sales': 'sum'
}).unstack().droplevel(0, axis=1)

# Heatmap temporal
plt.figure(figsize=(10, 6))
```

```

sns.heatmap(
    pivote_temporal,
    cmap='YlOrRd',
    linewidth=0.3,
    annot=True,
    fmt='.0f'
    # sin decimales, las cifras por década son
    ↪ grandes
)
plt.title('Ventas globales por Género y Década')
plt.show()

```



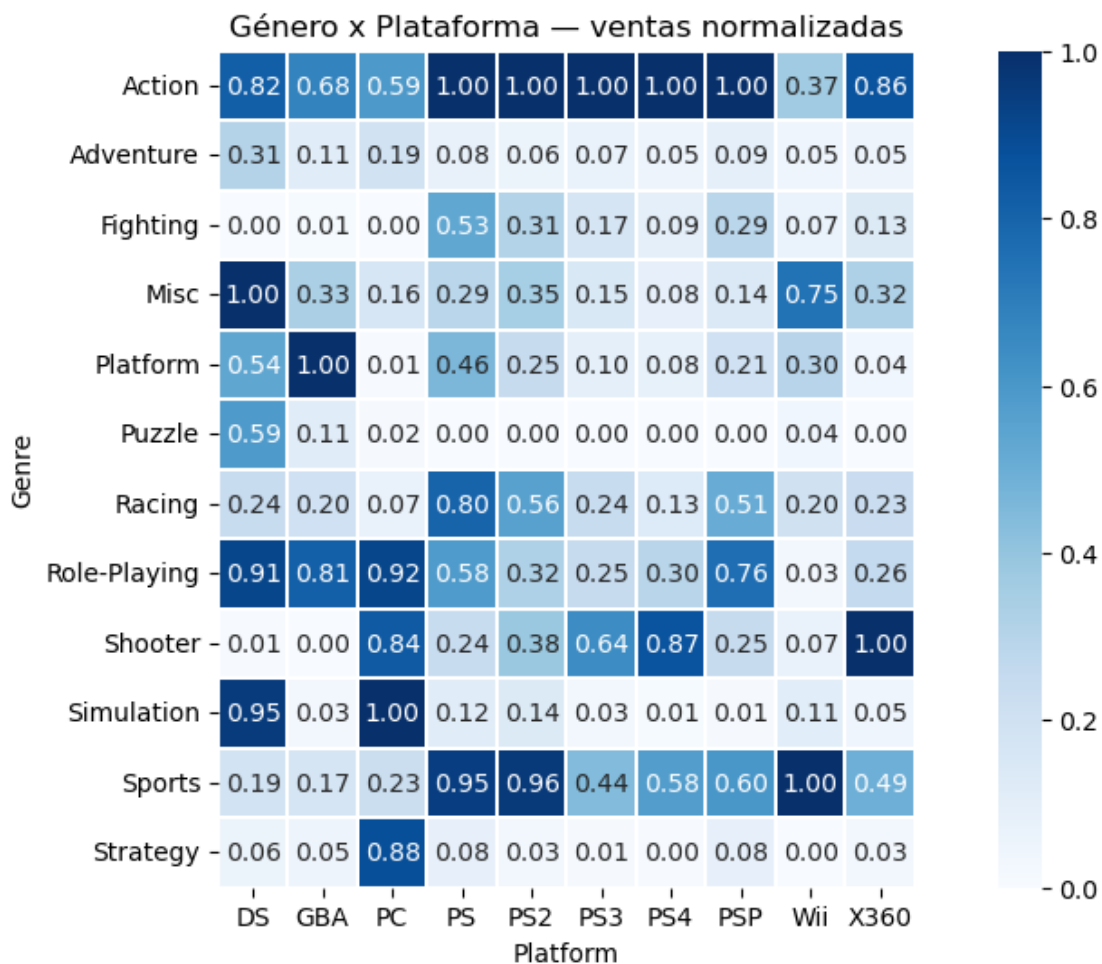
1.5 Paso 4: Heatmap Normalizado y Comparación con Barras

Objetivo: Normalizar la tabla pivote del Paso 1 para comparar géneros en igualdad de condiciones (independientemente de su volumen total de ventas), y comparar visualmente el heatmap con un gráfico de barras múltiples para la misma información.

Instrucciones: 1. Normaliza la tabla pivote del Paso 1 con Min-Max por columna (plataforma): $\text{pivote_norm} = (\text{pivote} - \text{pivote.min()}) / (\text{pivote.max()} - \text{pivote.min()})$. Rellena los NaN con 0 usando `.fillna(0)`. 2. Haz un heatmap estilizado con `plt.figure(figsize=(12, 6))`, `cmap='Blues'`, `square=True`, `linewidth=0.2` y `annot=True`, `fmt='.2f'`. Ponle un título descriptivo. 3. En una nueva celda, intenta representar la **misma información** (Genre × Platform × ventas) con `sns.barplot()` usando `hue='Platform'` y `x='Genre'`. Rota las etiquetas del eje X con `ax.tick_params(axis='x', labelrotation=45)`. 4. Compara ambos gráficos: ¿cuál transmite mejor los patrones de un vistazo cuando hay muchas categorías?

```
[7]: # Min-Max por columna (cada plataforma se compara consigo misma)
# Así PS2/X360 no aplastan visualmente al resto
pivote_norm = (pivote - pivote.min()) / (pivote.max() - pivote.min())
pivote_norm = pivote_norm.fillna(0)

# Heatmap de dos variables categóricas que codifican una variable numérica
plt.figure(figsize=(12, 6))
ax = sns.heatmap(
    pivote_norm,
    square=True, # Me genera cuadraditos simetricos
    cmap='Blues',
    linewidth=0.2,
    annot=True,
    fmt='.2f' # 2 decimales porque ahora todo está en [0,1]
)
plt.title('Género x Plataforma - ventas normalizadas')
plt.show()
```

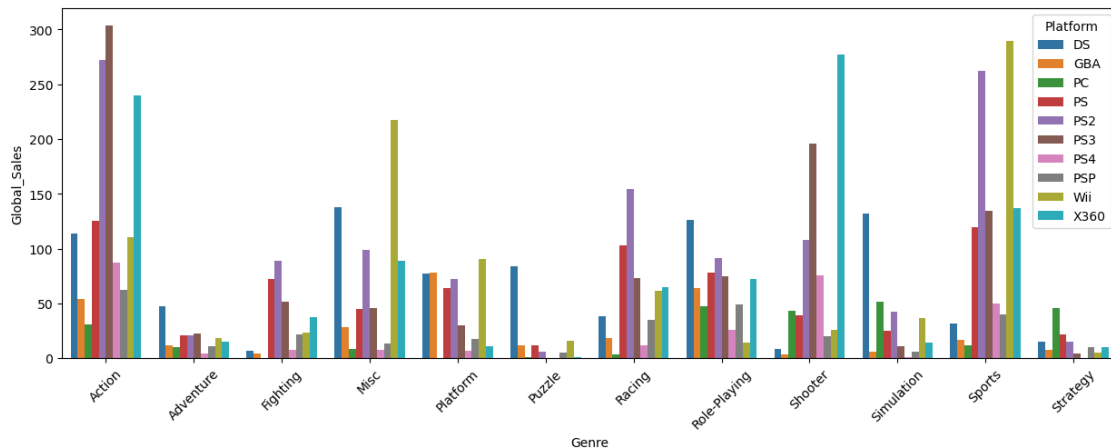



```
[8]: # Intentando hacer un grafico de barras multiples con la misma informacion
plt.figure(figsize=(15, 5))
temp_df = agrupado.reset_index()

ax = sns.barplot(
    data=temp_df,
    x='Genre',
    y='Global_Sales',
    hue='Platform',
    linewidth=0
)

# Rotacion de las etiquetas del eje X
ax.tick_params(axis='x', labelrotation=45)

plt.show()
```



1.5.1 Interpretación: Heatmap vs Gráfico de Barras

Ambos gráficos muestran **exactamente la misma información**, pero con resultados muy distintos:

- **El gráfico de barras múltiples** se vuelve muy difícil de leer cuando hay muchas categorías: las barras se amontonan, la leyenda se llena de colores y es complicado comparar la misma plataforma entre distintos géneros.
- **El heatmap** convierte esa misma tabla en un vistazo: el ojo humano detecta inmediatamente las celdas más oscuras (mayor venta) y los patrones por fila o columna, sin necesidad de leer valores uno a uno.
- **Regla general:** cuando tienes más de 4-5 categorías en cada eje, el heatmap gana al gráfico de barras múltiples.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **Heatmap Categórico** es la herramienta perfecta para explorar tablas de contingencia: cualquier cruce de dos variables categóricas con un valor numérico asociado es un candidato natural para este gráfico. * Recuerda dos reglas importantes: (1) si las variables tienen **escalas muy distintas** entre sí, normaliza los datos antes de hacer el heatmap o una variable dominará todo el color; (2) incluye siempre `annot=True` cuando el número de celdas sea manejable, ya que los valores exactos complementan el color. * Como reto adicional: ¿podrías crear un heatmap cruzando **Genre** con **Publisher** (filtrando solo los 10 publishers con más ventas) para ver qué editorial domina en cada género?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Cambio Platform por Publisher y ya. Apuesto a que Nintendo manda en Platform y Sports, y EA sube en Sports y Action.

```
[9]: # Reto: Género x top 10 Publishers (mismo esquema del paso 1 cambiando Platform_
    ↪ por Publisher)
top_publishers = df.groupby('Publisher')['Global_Sales'].sum().nlargest(10).
    ↪ index
df_pub = df[df['Publisher'].isin(top_publishers)]

# Agrupamos datos por género y publisher
agrupado_pub = df_pub.groupby(['Genre', 'Publisher']).agg({
    'Global_Sales': 'sum'
})

pivote_pub = agrupado_pub.unstack().droplevel(0, axis=1)

# Heatmap de variables categóricas que codifican una variable numérica
plt.figure(figsize=(12, 6))
sns.heatmap(
    pivote_pub,
    annot=True,
    fmt='.0f',
    linewidth=0.5,
    cmap='YlOrRd'
)
plt.title('Ventas globales por Género y top 10 Publishers')
plt.show()
```

